

NiceGUI 中 app 深度解析

在 NiceGUI 中，`app` 是框架的**核心应用实例对象**，它封装了 Web 应用的底层配置、生命周期管理、存储系统、路由规则、全局状态等关键能力，是连接 NiceGUI 前端交互与后端服务（基于 FastAPI/Starlette + WebSocket）的核心枢纽。开发者通过操作 `app` 实例，可实现对应用的全局配置、自定义扩展和高级功能开发。本文将从**核心定位、初始化与配置、核心功能模块、生命周期管理、高级扩展**五个维度，对 `app` 类进行全方位的详细阐述。

一、app 类的核心定位

NiceGUI 的 `app` 实例是**单例对象**（框架初始化时自动创建，无需手动实例化），其核心定位可总结为：

- 应用总控中心**：管理应用的启动、运行、停止，配置服务端口、域名、HTTPS 等基础参数；
- 存储系统入口**：提供 `app.storage` 系列接口，实现会话、用户数据的持久化与临时存储；
- 路由管理枢纽**：维护页面路由规则，支持自定义路由、中间件注入，扩展 FastAPI 底层路由能力；
- 全局状态容器**：存储应用级的全局变量、配置项，实现跨会话、跨页面的全局状态共享；
- 底层服务桥接**：暴露 FastAPI 应用实例、WebSocket 连接池等底层资源，支持与 FastAPI/Starlette 生态的深度集成。

二、app 类的初始化与基础配置

NiceGUI 会在导入 `ui` 模块时自动初始化 `app` 实例，开发者可通过 `from nicegui import app` 直接导入并配置。其基础配置主要围绕**服务启动参数**和**应用全局设置**展开。

2.1 启动参数配置

`app` 实例的启动参数可通过 `ui.run()` 间接传递（推荐），也可直接修改 `app` 的属性配置，核心参数包括：

参数名	作用	示例
<code>port</code>	服务监听端口	<code>ui.run(port=8080)</code> 或 <code>app.port = 8080</code>
<code>host</code>	服务监听地址 (0.0.0.0 表示外网可访问)	<code>ui.run(host='0.0.0.0')</code>
<code>title</code>	浏览器标签页标题	<code>ui.run(title='My NiceGUI App')</code>
<code>ssl_certfile / ssl_keyfile</code>	HTTPS 证书与私钥路径	<code>ui.run(ssl_certfile='cert.pem', ssl_keyfile='key.pem')</code>
<code>reload</code>	开发模式热重载	<code>ui.run(reload=True)</code> （仅开发环境）
<code>uvicorn_log_level</code>	底层 Uvicorn 日志级别	<code>ui.run(uvicorn_log_level='info')</code>

示例：直接配置 app 属性

```

from nicegui import ui, app

# 直接修改 app 实例的基础配置
app.port = 8081
app.title = 'Custom NiceGUI App'
app.host = '0.0.0.0'

@ui.page('/')
def index():
    ui.label('自定义 app 配置示例')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run() # 无需传递参数，使用 app 的已配置属性

```

2.2 全局设置配置

`app` 实例提供了多个属性用于配置应用的全局行为，常见的有：

1. `app.config`：框架核心配置，如是否启用深色模式、WebSocket 心跳间隔等；
2. `app.static_files`：配置静态文件目录（如图片、CSS、JS）；
3. `app.middleware`：注入 FastAPI 中间件，实现请求拦截、日志记录等；
4. `app.templates`：配置自定义 HTML 模板，扩展前端页面结构。

示例：配置静态文件目录

```

from nicegui import ui, app
from pathlib import Path

# 配置静态文件目录：将 ./static 目录映射为 /static 路由
app.add_static_files('/static', Path(__file__).parent / 'static')

@ui.page('/')
def index():
    # 访问静态文件：./static/images/logo.png
    ui.image('/static/images/logo.png')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

三、app 类的核心功能模块

`app` 实例的核心能力集中在**存储系统**、**路由管理**、**中间件**、**全局状态**四大模块，这也是开发者最常使用的功能。

3.1 存储系统：app.storage

`app.storage` 是 NiceGUI 提供的**分层存储接口**，分为会话存储、用户存储和全局存储，是实现数据持久化、跨会话共享的核心，与前文的 `ui.session` 紧密配合。

3.1.1 存储分层说明

存储类型	接口	存储位置	隔离性	持久化	适用场景
会话存储	<code>app.storage.session</code>	服务端内存	按 Session ID 隔离	服务重启丢失	临时会话数据（如未登录的临时筛选条件）
用户存储	<code>app.storage.user</code>	本地 JSON 文件（默认路径： <code>~/.nicegui/storage</code> ）	按用户标识隔离	永久保存（除非手动删除）	持久化用户配置（如“记住我”、个性化设置）
全局存储	<code>app.storage.general</code>	本地 JSON 文件	无隔离（应用级）	永久保存	应用全局配置（如系统参数、版本号）

3.1.2 存储接口的使用示例

```
from nicegui import ui, app

@ui.page('/')
def index():
    # 1. 会话存储：按 Session ID 隔离，内存存储
    app.storage.session[ui.session.id]['temp_data'] = '临时会话数据'
    ui.label(f"会话存储数据: {app.storage.session[ui.session.id].get('temp_data')}")

    # 2. 用户存储：持久化，按用户标识隔离（默认用 Session ID 作为用户标识）
    app.storage.user['username'] = '张三' # 自动保存到 JSON 文件
    app.storage.user['theme'] = 'dark'
    ui.label(f"用户存储数据: {app.storage.user.get('username')} | {app.storage.user.get('theme')}")

    # 3. 全局存储：应用级共享，持久化
    app.storage.general['app_version'] = '1.0.0'
    app.storage.general['max_users'] = 100
    ui.label(f"全局存储数据: {app.storage.general.get('app_version')} | {app.storage.general.get('max_users')}")

    # 清除存储数据
    def clear_storage():
        app.storage.session.pop(ui.session.id, None) # 清除当前会话的存储
        app.storage.user.clear() # 清除当前用户的持久化数据
        ui.notify('存储数据已清除', type='warning')
```

```
ui.button('清除存储', on_click=clear_storage)

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

关键说明：`ui.session` 是 `app.storage.session` 的**上下文友好封装**，日常开发中优先使用 `ui.session` 操作会话数据，仅在跨上下文访问时（如异步任务）使用 `app.storage.session`。

3.2 路由管理：app.router

NiceGUI 的页面路由基于 FastAPI 实现，`app.router` 暴露了底层的路由能力，支持**自定义路由**、**动态路由**、**路由拦截**，补充了 `@ui.page` 装饰器的基础路由功能。

3.2.1 核心路由操作

1. **添加自定义 API 路由：**基于 FastAPI 的接口，实现后端接口开发；
2. **动态注册页面：**运行时动态添加页面路由，适用于插件化开发；
3. **路由中间件：**拦截路由请求，实现鉴权、日志、跨域等功能。

3.2.2 路由使用示例

```
from nicegui import ui, app
from fastapi import Request, Response

# 1. 添加自定义 API 路由（基于 FastAPI）
@app.router.get('/api/get_user')
async def get_user(request: Request):
    """自定义 FastAPI 接口，返回用户数据"""
    return {
        'username': app.storage.user.get('username', 'guest'),
        'session_id': request.cookies.get('nicegui-session')
    }

# 2. 动态注册页面
def dynamic_page():
    ui.label('这是动态注册的页面！')

app.router.add_page('/dynamic', dynamic_page) # 等价于 @ui.page('/dynamic')

# 3. 路由中间件：拦截所有请求，记录日志
@app.router.middleware('http')
async def log_middleware(request: Request, call_next):
    print(f"请求路径: {request.url.path} | 客户端IP: {request.client.host}")
    response = await call_next(request)
    return response

@ui.page('/')
def index():
    ui.label('路由管理示例')
    # 调用自定义 API
    async def fetch_user():
        resp = await ui.request.get('/api/get_user')
```

```

        ui.notify(f"API 返回: {resp.json()}", type='info')

    ui.button('调用自定义 API', on_click=fetch_user)
    ui.button('前往动态页面', on_click=lambda: ui.navigate.to('/dynamic'))

if __name__ in {'__main__', '__mp_main__':
    ui.run()

```

3.3 中间件: app.middleware

中间件是**请求 / 响应的拦截器**，可在请求到达页面 / 接口前、响应返回客户端前执行自定义逻辑。`app.middleware` 直接封装了 FastAPI 的中间件能力，支持 `http`、`websocket` 两种类型的中间件。

示例：鉴权中间件（拦截所有请求）

```

from nicegui import ui, app
from fastapi import Request, HTTPException

# 全局鉴权中间件：拦截所有 HTTP 请求
@app.middleware('http')
async def auth_middleware(request: Request, call_next):
    # 排除登录页和公开接口
    if request.url.path in ['/login', '/api/public']:
        return await call_next(request)
    # 检查会话中的登录状态
    session_id = request.cookies.get('nicegui-session')
    if not session_id or not app.storage.session.get(session_id, {}).get('is_login'):
        raise HTTPException(status_code=307, detail='Redirect to login', headers=
{'Location': '/login'})
    return await call_next(request)

@ui.page('/login')
def login():
    def do_login():
        app.storage.session[ui.session.id]['is_login'] = True # 标记登录状态
        ui.navigate.to('/protected')

    ui.button('模拟登录', on_click=do_login)

@ui.page('/protected')
def protected():
    ui.label('受保护的页面，仅登录后可访问')

if __name__ in {'__main__', '__mp_main__':
    ui.run()

```

3.4 全局状态: app.state

`app.state` 是 NiceGUI 提供的**应用级全局状态容器**，用于存储跨会话、跨页面的全局变量，其数据在应用运行期间持续存在（服务重启丢失）。

示例：全局计数器（跨会话共享）

```
from nicegui import ui, app

# 初始化全局状态：跨所有会话共享的计数器
if not hasattr(app.state, 'counter'):
    app.state.counter = 0

@ui.page('/')
def index():
    # 读取全局计数器
    ui.label(f"全局计数器: {app.state.counter}").bind_text_from(app.state, 'counter')

    # 增加计数器
    def increase():
        app.state.counter += 1

    # 重置计数器
    def reset():
        app.state.counter = 0

    ui.button('增加', on_click=increase)
    ui.button('重置', on_click=reset)

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

关键说明： `app.state` 是非持久化的，服务重启后数据丢失；若需持久化全局状态，应使用 `app.storage.general`。

四、app 类的生命周期管理

`app` 实例的生命周期与 Web 应用的运行周期一致，分为**初始化**、**启动**、**运行**、**停止**四个阶段，NiceGUI 提供了钩子函数用于在生命周期节点执行自定义逻辑。

4.1 生命周期钩子

钩子函数	触发时机	适用场景
<code>app.on_startup</code>	应用启动前（Uvicorn 服务启动后）	初始化数据库连接、加载配置文件
<code>app.on_shutdown</code>	应用停止前（Uvicorn 服务停止前）	关闭数据库连接、保存临时数据
<code>app.on_connect</code>	客户端首次连接时（WebSocket 建立）	初始化会话、记录客户端信息
<code>app.on_disconnect</code>	客户端断开连接时（WebSocket 关闭）	清理会话数据、统计在线人数

4.2 生命周期钩子使用示例

```
from nicegui import ui, app
from datetime import datetime

# 1. 启动钩子：初始化应用
```

```

@app.on_startup
async def on_startup():
    print(f"应用启动于: {datetime.now()}")
    # 初始化全局存储
    if not app.storage.general.get('start_time'):
        app.storage.general['start_time'] = str(datetime.now())

# 2. 停止钩子: 清理资源
@app.on_shutdown
async def on_shutdown():
    print(f"应用停止于: {datetime.now()}")
    # 保存停止时间
    app.storage.general['stop_time'] = str(datetime.now())

# 3. 连接钩子: 客户端连接时触发
@app.on_connect
async def on_connect(client: app.Client):
    print(f"客户端 {client.id} 已连接 | Session ID: {client.session.id}")
    # 初始化会话计数器
    if not ui.session.get('visit_count'):
        ui.session['visit_count'] = 0

# 4. 断开连接钩子: 客户端断开时触发
@app.on_disconnect
async def on_disconnect(client: app.Client):
    print(f"客户端 {client.id} 已断开 | Session ID: {client.session.id}")

@ui.page('/')
def index():
    # 统计访问次数
    ui.session['visit_count'] += 1
    ui.label(f"当前会话访问次数: {ui.session['visit_count']}")
    ui.label(f"应用启动时间: {app.storage.general.get('start_time')}")

if __name__ in {'__main__', '__mp_main__':
    ui.run()

```

五、app 类的高级扩展

`app` 实例暴露了 NiceGUI 的底层资源，支持与 FastAPI/Starlette 生态、第三方库的深度集成，实现高级扩展功能。

5.1 集成 FastAPI 生态

NiceGUI 基于 FastAPI 开发，`app.native` 直接暴露了底层的 FastAPI 应用实例，可无缝集成 FastAPI 的插件（如 `fastapi-users`、`fastapi-jwt-auth`）。

示例：集成 FastAPI 的 Swagger 文档

```

from nicegui import ui, app
from fastapi import FastAPI

# 获取底层 FastAPI 实例

```

```

fastapi_app: FastAPI = app.native

# 添加 FastAPI 接口, 自动生成 Swagger 文档
@fastapi_app.get('/api/hello')
def hello(name: str = 'world'):
    return {'message': f'Hello {name}!'}

@ui.page('/')
def index():
    ui.label('集成 FastAPI Swagger 文档示例')
    # 跳转到 FastAPI 的 Swagger 文档页 (/docs)
    ui.button('查看 API 文档', on_click=lambda: ui.navigate.to('/docs'))

if __name__ in {'__main__', '__mp_main__':
    ui.run()

```

说明：启动应用后，访问 `/docs` 即可看到 FastAPI 自动生成的 Swagger 接口文档。

5.2 自定义 WebSocket 处理

NiceGUI 基于 WebSocket 实现前端与后端的实时通信，`app.websockets` 暴露了 WebSocket 连接池，可自定义实时通信逻辑。

示例：广播消息到所有客户端

```

from nicegui import ui, app
from starlette.websockets import WebSocket

@ui.page('/')
def index():
    msg_input = ui.input('输入广播消息')
    # 广播消息到所有连接的 WebSocket 客户端
    async def broadcast():
        message = msg_input.value.strip()
        if not message:
            return
        # 遍历所有 WebSocket 连接
        for websocket in app.websockets.values():
            if isinstance(websocket, WebSocket) and websocket.client_state.value ==
'connected':
                await websocket.send_text(f"广播消息: {message}")
        ui.notify('消息已广播', type='success')

    ui.button('广播', on_click=broadcast)
    # 接收 WebSocket 消息
    ui.chat_message('等待广播消息...', name='系统')

    # 注册 WebSocket 消息处理
    @app.on_websocket_message
    async def handle_websocket_message(websocket: WebSocket, message: str):
        ui.chat_message(message, name='广播')

if __name__ in {'__main__', '__mp_main__':

```



```
ui.run()
```

5.3 多应用实例与子应用

`app` 实例默认是单例的，但 NiceGUI 支持创建**子应用实例**，实现多模块、多租户的隔离部署。

示例：创建子应用

```
from nicegui import ui, App

# 创建主应用
main_app = App()

# 创建子应用
sub_app = App()

# 主应用页面
@main_app.page('/')
def main_index():
    ui.label('这是主应用!')
    ui.button('前往子应用', on_click=lambda: ui.navigate.to('/sub'))

# 子应用页面（挂载到 /sub 路由）
@sub_app.page('/')
def sub_index():
    ui.label('这是子应用!')

# 将子应用挂载到主应用的 /sub 路由
main_app.mount('/sub', sub_app)

if __name__ in {'__main__', '__mp_main__'}:
    main_app.run(port=8080)
```

六、app 类的注意事项与常见坑

6.1 单例特性与多线程

`app` 实例是单例的，在多线程 / 多进程环境中修改 `app` 的属性需注意线程安全，建议通过 `app.state` 或 `app.storage` 存储共享数据，避免直接修改 `app` 的实例属性。

6.2 存储路径的配置

`app.storage` 的默认存储路径为 `~/.nicegui/storage`，若需自定义路径，可在应用启动前设置 `app.storage.path`：

```
from nicegui import app
from pathlib import Path

app.storage.path = Path(__file__).parent / 'custom_storage' # 自定义存储路径
```

6.3 上下文限制

部分 `app` 的功能（如 `app.storage.user`）依赖 `ui.session` 的上下文，在脱离请求 / 事件上下文时（如启动钩子中）访问需注意是否有有效会话。

6.4 生产环境的部署

在生产环境中，`app` 的启动建议使用 Uvicorn 直接运行，而非 `ui.run()`，以获得更好的性能和可配置性：

```
# main.py
from nicegui import ui, app

@ui.page('/')
def index():
    ui.label('生产环境部署示例')

if __name__ == '__main__':
    import uvicorn
    uvicorn.run(app.native, host='0.0.0.0', port=8080)
```

七、总结

`app` 类是 NiceGUI 框架的**核心枢纽**，它封装了 Web 应用的所有底层能力，从基础的服务配置、路由管理，到高级的存储系统、生命周期钩子，再到与 FastAPI 生态的深度集成，都通过 `app` 实例对外暴露。

开发者通过 `app` 实例可实现：

1. **全局配置**：设置服务端口、HTTPS、静态文件目录等；
2. **数据存储**：利用 `app.storage` 实现会话、用户、全局数据的持久化；
3. **路由扩展**：添加自定义 API、动态页面、中间件；
4. **生命周期管理**：在应用启动 / 停止、客户端连接 / 断开时执行自定义逻辑；
5. **高级扩展**：集成 FastAPI 插件、自定义 WebSocket、创建子应用。

掌握 `app` 类的使用，是从 NiceGUI 基础开发走向高级定制的关键，结合前文的 `ui.session`，可完整实现 NiceGUI 应用的状态管理、鉴权控制、数据持久化等核心业务需求。